University of Engineering and Technology Lahore

Department of Electrical Engineering

Computer Architecture Lab Manual

Course Code: EE-475L

Spring 2026

Contents

Introduction

The goal of this lab is to familiarize students with the process of designing a RISC-V processor, beginning with a single-cycle implementation and then extending it into a pipelined architecture capable of handling control and data hazards. This course provides practical exposure to designing digital systems using RTL descriptions, resolving hazards in a simple pipeline, and building functional interfaces. Additionally, students will learn how to approach system-level optimization, bridging the gap between individual components and a fully integrated processor.

In tackling these challenges, your first step will be to map the high-level specification to a design that can be translated into a hardware implementation. Following that, you will produce and debug the implementation. These steps can take a significant amount of time if the design is not carefully planned before implementation.

By the end of this lab, students will have gained hands-on experience in processor design, hazard resolution, and system-level thinking, equipping them with essential skills for real-world digital system development. **We will basically follow this EECS151_SP24 document created by University of Berkeley to design the pipelined processor.**

Experiment 1

Design of Arithmetic Logic Unit and its Controllers

1.1 Overview

In this experiment, students will design and implement an ALU and its controller using SystemVerilog. The design will support RISC-V instructions and will be verified through simulation. This experiment serves as a foundational step toward building a complete RISC-V processor, as the ALU and its controller are critical components used throughout the datapath in later experiments.

1.2 ALU

The Arithmetic Logic Unit (ALU) is a fundamental component of a processor responsible for performing arithmetic and logical operations. These operations include addition, subtraction, logical AND/OR/XOR, comparison, and shift operations.

1.2.1 Block Diagram

Task 1: Draw the schematic / block diagram of the ALU shown in Figure 2.1 using draw.io.

The input and output port names are as shown in Figure 2.1. The ALU module takes two input operands and performs an operation specified by the control signal **alu_operation** and produces the **result**. In addition to the result, the ALU generates a **zero** flag, which is asserted when the result of a subtraction operation is zero.

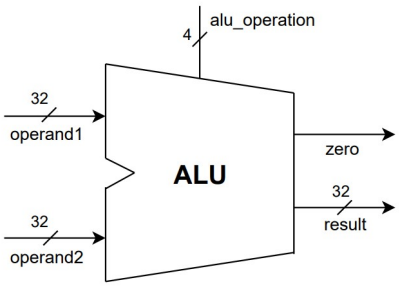


Figure 1.1: ALU

1.2.2 Understanding the RISCV ISA Cheat Sheet (Reference Manual)

Read and understand the first page of this Riscv Cheat Sheet. For more details you can refer to this reference manual.

1.3 Controller Design

The ALU controller determines which operation the ALU should perform for a given instruction. It takes different fields of the RISCV instruction such as **OPCODE, func3** etc. and translates them into a specific control code that drives the ALU. Before designing the controller, you should decide the names and size of the inputs and outputs of the controller. Figure 2.2 shows the controller you will be implementing. The output **alu_operation** depends on three inputs **func3**, **func7** and **ALUOp**.

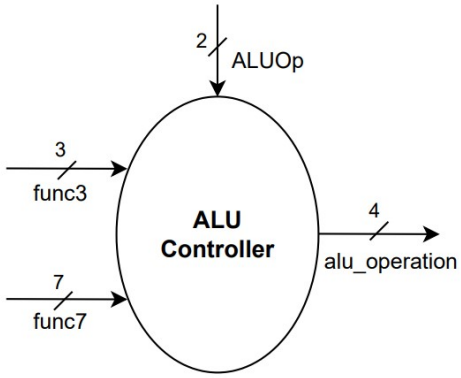


Figure 1.2: ALU Controller

Task 2: Look at the ISA cheat sheet and list down all the arithmetic, bitwise and shift operations in the first column of table 1.1, to be performed by the ALU. The operators must have at least one of `rs1, rs2 or imm` as one of the operands. Decide the values and size of the output **alu_operation** in binary yourself. Use WORD or EXCEL to complete the table. The values given in the table is just an example.

Operations	**alu_operation (Controller Output)**
and	0000
or	0001
add	0010

Table 1.1:

Task 3: The next task is to decide the values of the **ALUOp** signal. In figure 1.3 observe that **ALUOp** does not depend on the instruction type only. Use this strategy to complete table 1.2 for all the instructions.

Task 4: Draw block diagram of first level controller which has one input **Opcode** and one output **ALUOp**. Determine their size yourself.

Instruction opcode	ALUOp	Operation	Funct7 field	Funct3 field	Desired ALU action	ALU control input
lw	00	load word	XXXXXXX	XXX	add	0010
sw	00	store word	XXXXXXX	XXX	add	0010
beq	01	branch if equal	XXXXXXX	XXX	subtract	0110
R-type	10	add	0000000	000	add	0010
R-type	10	sub	0100000	000	subtract	0110
R-type	10	and	0000000	111	AND	0000
R-type	10	or	0000000	110	OR	0001

Figure 1.3: ALUOp Table

Instruction	func3	func7	ALUOp	Desired ALU Action
and	000	0000000	10	AND
sub				
xor				
or				
and				
slt				
sll				
sltu				
srl				
sra				
addi				
xori				
ori				
andi				
slti				
slli				
sltui				
srli				
srai				
lb/sb				
lh/sh				
lw/sw				
lbu				
lhu				
lui				
auipc				
jal				
jalr				
beq				
bne				
blt				
bge				
bltu				
bgeu				

Table 1.2:

1.4 Deliverables

- Complete tables 1.1 and 1.2.
- Block diagrams of both controllers.

Acknowledgement

This manual is the result of the work of many including

1. Rehan Naeem (2026)
2. Tayyba Shafiq (2026)

Experiment 2

Implementation of the ALU and its Controller in SystemVerilog

2.1 Overview

In this experiment, we will design and implement key processor functional units in SystemVerilog. We will implement the ALU and its controller in system verilog. We will also test the functionality of these modules by writing testbenches in system verilog.

Additionally, we will also implement the Immediate Generator. This lab provides hands-on experience in writing combinational logic, control logic, and verifying hardware modules, forming the core building blocks of a processor datapath.

2.2 ALU Implementation in SystemVerilog

In this task, students will implement the Arithmetic Logic Unit (ALU) in SystemVerilog. A block diagram of the ALU is provided in Figure 2.1.

Task1: Implementation Write a System Verilog code to implement the ALU module which in turn implements the operations in Table 1.1. Input and output port names should be same as shown in the block diagram. Feel free to use additional flags when needed. For example, the **zero** flag is 1 when both operands are equal.

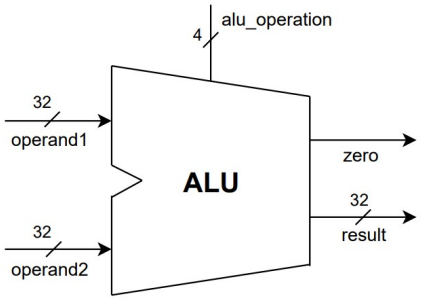


Figure 2.1: ALU

Task 2: Testing After implementing the ALU module, verify its functionality by writing a System Verilog randomized testbench. The testbench should apply a variety of input combinations and control signals to ensure that all arithmetic and logical operations produce the correct results.

2.3 ALU Controller Implementation

In this task, students will implement the ALU Controller in SystemVerilog.

Task 1: Implementation Implement the ALU Controller (second level controller) module provided in Figure 2.2 using the ALU operation table from Experiment 1 (see Table 1.2). Input/output port names should be same as shown in the block diagram.

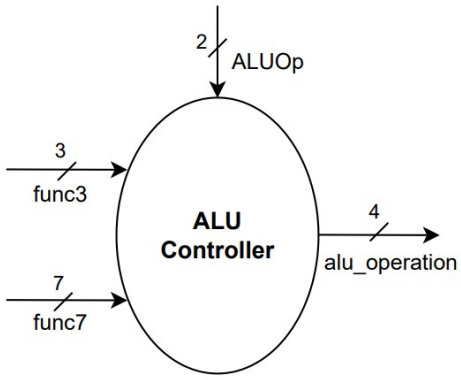


Figure 2.2: ALU Controller

Task 2: Write system verilog code for the first level controller which takes the opcode field of the instruction as input and outputs the ALUOp according to Table 1.2

Task 3: Testing Write a randomized testbench in SystemVerilog to verify the functionality of both the ALU controllers. The testbench should apply various instruction inputs and function codes to ensure that the controller generates the correct control signals for all ALU operations.

2.3.1 Use of `opcode.vh` File

While implementing the ALU controller in SystemVerilog, writing raw binary values for opcodes and function fields can easily lead to errors. To avoid such mistakes, students are provided with an opcode.vh header file containing predefined macros for all RISC-V opcodes and function codes.

Students are strongly encouraged to include this file in their SystemVerilog source files as shown below:

```
`include "opcode.vh"
```

After including the file, opcodes and function codes should be referenced using the defined macros instead of hard-coded binary values.

Example: To check for a load instruction, use:

```
if (opcode == `OPC_LOAD) begin
...
end
```

Similarly, other instructions can be identified using macros such as `` `OPC_BRANCH ``, `` `OPC_JAL ``, and `` `OPC_LUI ``. This approach improves code readability, reduces errors, and improves maintainability of the design.

2.4 Deliverables

- RTL and testbench codes for ALU and the two controllers.

Acknowledgement

This manual is the result of the work of many including

1. Rehan Naeem (2026)
2. Tayyba Shafiq (2026)

Experiment 3

Implementation of Immediate Generator in SystemVerilog

3.1 Overview

In this lab, we will design and implement an Immediate Generator (ImmGen) module in SystemVerilog for a RISC-V processor. The Immediate Generator extracts and sign-extends immediate values according to the RISC-V encoding rules.

We will study how to implement combinational logic to generate the correct 32-bit immediate output. The functionality of the module will be verified through simulation and testbench development.

3.2 Immediate Generator

The Immediate Generator (ImmGen) is a key combinational block in the RISC-V datapath. Its purpose is to extract the immediate field from a 32-bit instruction and generate a correctly formatted 32-bit immediate value that can be used by the ALU or other datapath components.

In RISC-V, different instruction formats (I-type, S-type, B-type, U-type, and J-type) encode immediate values using different bit fields of the instruction as observed from cheatsheet. Therefore, the Immediate Generator must extract the appropriate immediate bit fields, rearrange bits when required, perform sign-extension or zero-extension if required and produce a final 32-bit immediate value.

3.2.1 Block Diagram

The Immediate Generator takes a 32-bit **instruction** as input and produces a 32-bit **immediate** as output.

Task 1: Draw the block diagram of the Immediate Generator using draw.io. Label all signals properly and mention the bit-width of each input and output field.

3.2.2 Implementation

Task 2: Write a SystemVerilog module to implement the Immediate Generator, which takes a 32-bit **instruction** as input and produces a 32-bit **immediate** as output. The

module must support all instruction formats. Refer to the Riscv Cheat Sheet for immediate field decoding. Input/output port names should be the same as shown in Figure 3.1.

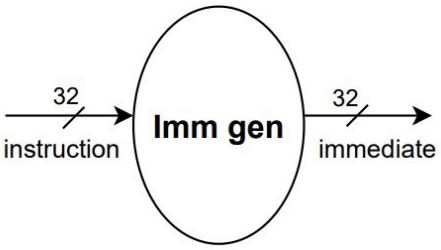


Figure 3.1: Immediate Generator Block Diagram

3.2.3 Testing

Task 3: Write a randomized testbench to verify the functionality of the Immediate Generator. Test the module using instructions from all supported formats and verify that the generated 32-bit immediate matches the expected value.

3.3 Deliverables

- Block diagram of Immediate Generator.
- RTL and testbench codes for Immediate Generator.

Acknowledgement

This manual is the result of the work of many including

1. Rehan Naeem (2026)
2. Tayyba Shafiq (2026)

Experiment 4

Implementation of R and I Type RISCV Instructions - Part 1

4.1 Overview

In this experiment, the RISC-V datapath is implemented to execute a subset of instructions from the RISC-V instruction set architecture (ISA). The focus of this lab is on designing and verifying the datapath required to support R-type and I-type instructions. These instruction formats are fundamental to RISC-V and are widely used for arithmetic, logical, and immediate-based operations.

The objective of this experiment is to develop a clear understanding of instruction decoding, operand selection, ALU operations, and data flow within a processor datapath, while reinforcing concepts related to computer architecture and digital hardware design.

4.2 Datapath Components

Read chapter 4 of the textbook Patterson Hennesy Risc5 for more details.

4.2.1 Program Counter

The `Program Counter` (PC) is a register that stores the address of the next instruction to be executed. It supplies this address to the instruction memory to fetch the instruction. The PC updates its value on every clock cycle and can be reset to the starting address during initialization.

4.2.2 Instruction Memory

The `Instruction Memory` stores the instructions that are executed by the processor. It receives the instruction address from the Program Counter (PC) and outputs the corresponding 32-bit instruction.

4.2.3 Register File

The `Register File` stores the processor's general-purpose registers and allows reading and writing of register values during instruction execution. It typically supports two read ports and one write port, enabling the processor to read two operands and write back.

In RISC-V, the register file contains 32 registers, each 32 bits wide, and register x0 is always hardwired to zero.

The code for resgister file is provided ASYNC_RAM_1W2R. Modify the parameters according to the specifications.

4.2.4 ALU

The ALU performs all arithmetic and logical operations in the processor. It takes two 32-bit operands and a control signal that specifies the operation. The ALU produces a 32-bit result and a zero flag indicating if the result is zero.

4.2.5 Data Memory

Read article 4.5 and 4.6 related to RAMs in EECS151_SP24. You have to implement the BIOS, IMEM and DMEM memories.

4.2.6 The First-Level Controller

The first-level controller generates the main control signals for the datapath based on the instruction opcode. These control signals determine how different components of the datapath operate, such as enabling register writes, selecting ALU inputs, and controlling memory operations. A reference table is given in Figure 4.1. Draw a table similar to the one shown below for your design.

Instruction	ALUSrc	Memto-Reg	Reg-Write	Mem-Read	Mem-Write	Branch	ALUOp1	ALUOp0
R-format	0	0	1	0	0	0	1	0
lw	1	1	1	1	0	0	0	0
sw	1	X	0	0	1	0	0	0
beq	0	X	0	0	0	1	0	1

Figure 4.1: Instruction Controller

4.2.7 ALU Controller

The ALU Controller determines the specific ALU operation to perform. It uses the instruction fields along with the ALU control input from the main controller, to generate the final alu_control signal that selects the required ALU operation.

4.3 R-Type Implementation

R-type instructions in RISC-V are used for register-to-register operations. Refer to Riscv Cheat Sheet for more details of R-type instruction fields.

Task 1: Design the datapath for R-type instructions. Decide which components are needed (e.g., Register File, ALU, PC, multiplexers) and the way they should be connected to correctly execute R-type instructions.

Draw the complete datapath for R-type Instructions using draw.io. Label all inputs and outputs consistent with your design. Show all controller signals clearly.

Task 2: To verify the datapath, students should hardcode the assembly instructions for all supported R-type instructions into the Instruction Memory. These instructions will then be executed by the datapath during simulation. The outputs should be observed to confirm that the correct operations are performed and the results are written back to the destination registers correctly.

4.4 I-Type Implementation

I-type instructions in RISC-V are used for immediate-based operations. Refer to Riscv Cheat Sheet for details of I-type instruction fields.

Task 3: Use your R-type datapath as the starting point and modify it to support I-type instructions.Decide which components need to be added or modified. Connect them properly so the datapath can execute I-type instructions. The execution of R-type instructions should not fail as a result.

Draw the complete datapath for I-type Instructions using draw.io. Label all inputs and outputs consistent with your design. Show the controller signals clearly if required.

Task 4: To verify the datapath, students should hardcode the assembly instructions for all supported I-type instructions into the Instruction Memory. The outputs should be observed to confirm that the correct operations are performed and the results are written back to the destination registers correctly.

Additionally, students should create a mixed instruction sequence containing both R-type and I-type instructions to ensure that the datapath correctly supports and executes both instruction types within the same program.

You can use this VENUS tool to simulate assembly programs.

4.5 Deliverables

- Block diagram of complete datapath
- RTL for all modules
- Testbench

Acknowledgement

This manual is the result of the work of many including

1. Rehan Naeem (2026)
2. Tayyba Shafiq (2026)

Experiment 5

Implementation of S, B, J and U Type RISCV Instructions - Part 2

4.1 Overview

n this experiment, we extend our understanding of the RISC-V instruction set by implementing the remaining instruction formats: S-type, B-type, J-type, and U-type instructions. In the previous experiment, we focused on R-type and I-type instructions, which primarily handle arithmetic, logical, and immediate operations. This experiment shifts the focus toward control flow and memory interaction, which are essential for building a functional processor.

Through this experiment, students will implement the datapath and control logic required to support these instruction types in a RISC-V processor. This includes modifying the instruction decoding mechanism, immediate generation, control signals, and updating the program counter logic for branching and jumping.

4.2 S-Type Implementation

S-type instructions in RISC-V are used for store operations, where data from a register is written to memory. Refer to Riscv Cheat Sheet for more details of S-type instruction fields.

Task 1: Design the datapath for S-type instructions. Decide which components are required (e.g., Register File, ALU, Data Memory, PC, Immediate Generator, multiplexers) and how they should be connected to correctly execute store operations.

Draw the complete datapath for S-type Instructions using draw.io. Label all inputs and outputs consistent with your design. Show all control signals clearly.

Task 2: To verify the datapath, students should hardcode assembly instructions for supported S-type instructions into the Instruction Memory. These instructions will then be executed by the datapath during simulation.

The outputs should be observed to confirm that the correct data is written to the appropriate memory locations based on the computed address.

4.3 B-Type Implementation

B-type instructions in RISC-V are used for conditional branching, allowing the program to change its execution flow based on comparisons between register values. Refer to Riscv Cheat Sheet for more details of B-type instruction fields.

Task 1: Design the datapath for B-type instructions. Identify the required components (e.g., Register File, ALU, PC, Immediate Generator, multiplexers) and the connections needed to evaluate branch conditions correctly.

Draw the complete datapath for B-type Instructions using draw.io. Label all inputs and outputs consistently, and show all control signals clearly, including the branch control logic.

Task 2: To verify the datapath, students should hardcode assembly instructions for supported B-type instructions into the Instruction Memory. During simulation, observe the program counter to ensure that branches are taken or not taken correctly based on the condition results.

The outputs should confirm that the datapath correctly handles conditional branching and updates the PC appropriately.

4.4 J-Type Implementation

J-type instructions in RISC-V are used for unconditional jumps, allowing the program to jump to a target address while optionally saving the return address in a register. Refer to Riscv Cheat Sheet for more details of J-type instruction fields.

Task 1: Design the datapath for J-type instructions. Identify the necessary components (e.g., Register File, PC, Immediate Generator, multiplexers) and their interconnections to correctly execute jump operations.

Draw the complete datapath for J-type Instructions using draw.io. Label all inputs and outputs consistently, and show all control signals clearly, including the PC update logic and register write-back control.

Task 2: To verify the datapath, students should hardcode assembly instructions for supported J-type instructions into the Instruction Memory. During simulation, observe both the updated PC and the destination register to ensure that jumps are executed correctly and the return addresses are stored properly.

The outputs should confirm that the datapath correctly implements unconditional jumps updating the program counter and storing return addresses as required for proper program flow.

4.5 U-Type Implementation

U-type instructions in RISC-V are used to load upper immediate values into registers, which is useful for generating large constants and computing addresses. Refer to Riscv Cheat Sheet for more details of U-type instruction fields.

Task 1: Design the datapath for U-type instructions. Identify the required components (e.g., Register File, PC, Immediate Generator, multiplexers) and their interconnections to handle upper immediate operations.

Draw the complete datapath for U-type Instructions using draw.io. Label all inputs and outputs consistently, and show all control signals clearly, including the ALU operation

and register write-back control.

Task 2: To verify the datapath, students should hardcode assembly instructions for supported U-type instructions into the Instruction Memory. During simulation, observe the destination register to confirm that the immediate values are loaded or added to the PC correctly. The outputs should verify that the datapath correctly implements U-type instructions.

You can use this VENUS tool to simulate assembly programs.

4.6 Deliverables

- Block diagram of complete datapath
- RTL for all modules
- Testbench

Acknowledgement

This manual is the result of the work of many including

1. Rehan Naeem (2026)
2. Tayyba Shafiq (2026)

Chapter 5

Guidelines

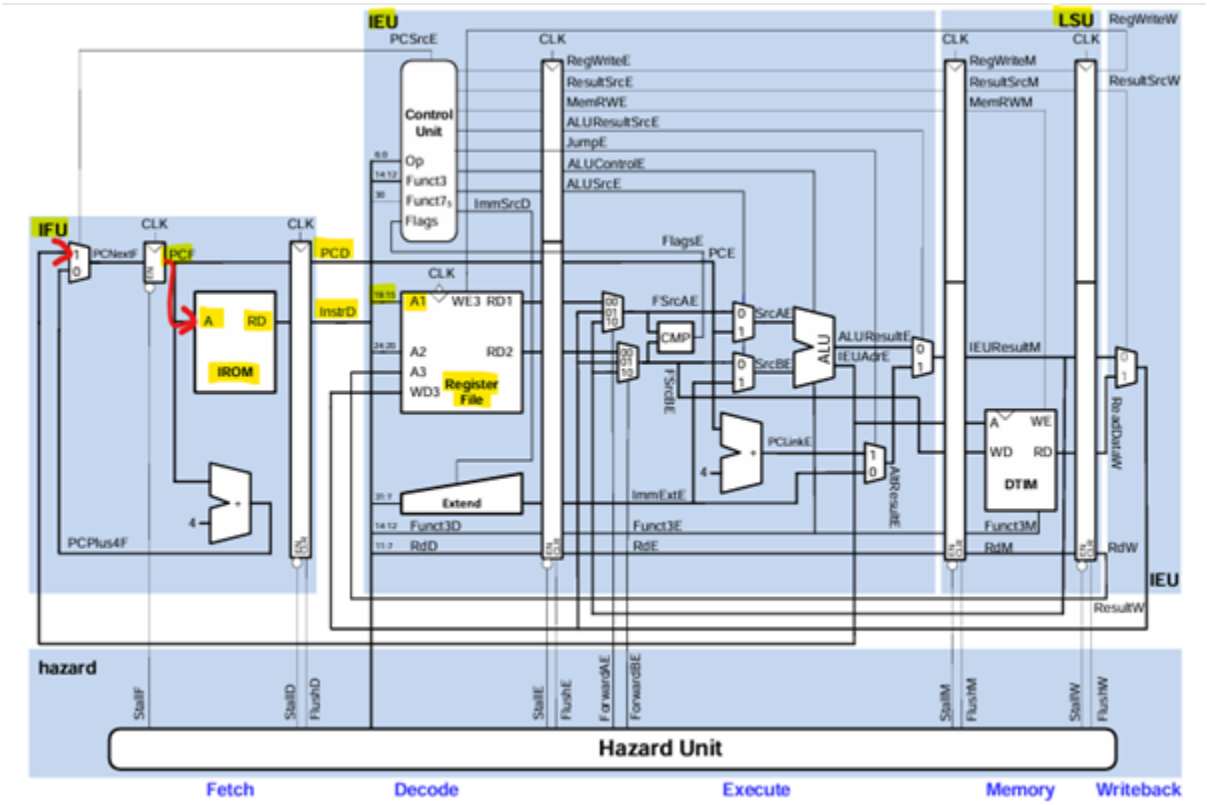
Below are some guidelines which you should strictly follow in this lab.

5.1 Specs/Requirements:

This will usually be given.

5.2 Block and Timing Diagrams:

- Highlight different (main) parts (comprising of blocks) of your design in light blue color. In the figure below, different parts like IFU, IEU, LSU are highlighted in light blue.
- Use an arrow to show inputs, outputs of a block and connections between the blocks. Arrow pointing towards a port represents input port and arrow pointing away from the port represents the output port. For example, PCF is the output port while A is the input port.
- Label the name of each block like IROM, Register File in the figure below. Label the name and size of each input and output port in the design like A and RD in IROM block. To label size, use the format <signal name>[MSB index : LSB index]. For example, PCD is a 32-bit signal so you can label it as PCD[31:0]. Sliced signals can be shown using the format MSB index : LSB index. For example, the 32-bit signal InstrD is sliced and its bits 15 to 19 go to A1 port of the Register File.



5.3 Code-Related:

- **File Naming:** Give proper names to the design and testbench files. The extension must be **sv** for both. Testbench file name must have the same name as that of the file it tests but must end with _tb. For example, `xyz.sv` could be the name of a design file while `xyz_tb.sv` would be the name of the testbench file. Module name must match the file name.
- **Files Organization:** Create one folder for each lab experiment and name it `lab x` where `x` is lab number. Inside `lab x` folder, create four folders `src`, `sim`, `diagrams`, and `questa`. Place design files in `src` folder, testbench files in `sim` folder, block, timing and waveform diagrams in png or jpg format in `diagrams` folder. Create new Questasim project in `questa` folder. Vivado files can be placed in the default path for them. Upload all the things except vivado and questa files to Github.
- **Style:** Coding style for the design is shown below. Look at the indentation, comments, parameter, input/output ports, internal wires declarations and module instantiations.

```
/*
This module does something
Look how the module is described.
Its a multi-line comment
*/
module xyz
#(
        parameter  N = 5,
        parameter  A = 15
)
(
        input  logic         [15:0] in1,
        input  logic                in2,
        output logic signed [3:0]  o
);

// defining internal wires. This is a one-line comment

wire logic [3:0] x1;   //x1 connects d and q ports of ff module
wire logic       x20;

//instantiation of ff and abc modules

ff  #(N(3)) dut1 (.d(x1),
                  .q(x1),
                  .rst(in2)
                 );

abc dut2 (.x(x20),
          .y(o)
         );

//always block doing something

always_ff @(posedge in1) begin

        if (in1 ==0) begin
                in1 <= 0;
        end
        else begin
                in1 <= 1;
        end

end

endmodule
```

Listing 5.1: Full Adder Testbench

5.4 QuestaSim Related

If you are in simulation mode in QuestaSim.... When you modify the testbench or the RTL file, you will observe green ticks convert to question mark. Recompile

5.5 Vivado Related

5.6 Misc

Use the following format when working with truth tables. Separate input ports and output ports by a vertical line as shown below.

Component Names	SW0	SW1	SW2	LD0	LD1	CA
FPGA Pin Names	J15	M16	L13	H17	K15	T10
Port Names	A	B	Cin	Sum	Cout	Res
	0	0	0	0	0	0
	0	0	1	1	0	1